

Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ

к лабораторной работе №1
на тему

**СИММЕТРИЧНАЯ КИПТОГРАФИЯ.
СТАНДАРТ ШИФРОВАНИЯ ГОСТ 28147-89**

БГУИР КП 6-05-0612-02 016 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф. Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение	3
1 Выполнение работы	4
Вывод	6
Приложение А (обязательное) Листинг программного кода	7

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается симметричный блочный алгоритм шифрования ГОСТ 28147-89. В соответствии с заданием требуется реализовать программные средства шифрования и дешифрования текстовых файлов в четырех режимах: простой замены, гаммирования, гаммирования с обратной связью, а также реализовать генерацию имитоприставки. ГОСТ 28147-89 использует 64-битный блок данных и 256-битный ключ, который разбивается на восемь 32-битных подключей. Обработка каждого блока выполняется последовательностью раундов с использованием модульного сложения, нелинейной подстановки и циклического сдвига.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для обработки текстовых файлов по алгоритму ГОСТ 28147-89. Реализация выполнена в операционной системе Ubuntu 24.04 LTS с использованием стандартной библиотеки C++ и системы сборки CMake. Программа организована модульно: отдельный модуль содержит реализацию базовых криптографических преобразований ГОСТ 28147-89, а главный модуль отвечает за разбор аргументов командной строки, обработку файлов и запуск выбранного режима.

В качестве основного алгоритма был реализован 32-раундовый цикл ГОСТ 28147-89. В начале 256-битный ключ разбивается на восемь 32-битных подключей. На каждом раунде правая часть блока складывается с текущим подключом по модулю 2^{32} , после чего результат проходит побайтовую или поблочную замену по таблице S-блоков. Полученное значение циклически сдвигается влево на 11 битов и складывается с другой половиной блока посредством операции XOR. Такой раунд повторяется в соответствии с установленной последовательностью использования подключей. При расшифровании используется обратный порядок подключей, что соответствует описанию алгоритма в методических материалах.

Для режима простой замены каждый 64-битный блок исходного файла обрабатывается независимо от остальных блоков одним и тем же ключом. При шифровании для последнего неполного блока используется дополнение, после чего каждый полный блок передается в процедуру шифрования ГОСТ 28147-89. При расшифровании блоки обрабатываются обратным преобразованием, после чего дополнение удаляется. Такой способ позволяет корректно обрабатывать текстовые файлы произвольной длины.

В режиме гаммирования для формирования гаммы используется 64-битная синхропосылка. Она шифруется базовым алгоритмом, а полученный результат используется как гамма для побитовой операции XOR с очередным участком открытого текста. Следующий участок обрабатывается уже следующим значением гаммы. Данный режим позволяет обрабатывать сообщение произвольной длины без добавления заполнения к последнему участку данных.

В режиме гаммирования с обратной связью начальное значение регистра также задается синхропосылкой. После обработки очередного блока результат шифрования используется в качестве нового состояния регистра при обработке следующего блока. При расшифровании для обратной связи используется соответствующий предыдущий блок шифртекста. Таким образом, преобразование последующего блока зависит от предыдущего шифртекста.

Для контроля целостности сообщения дополнительно реализована функция генерации имитоприставки. В реализованном программном средстве исходные данные разбиваются на 64-битные части, над ними последовательно выполняются операции с использованием алгоритма ГОСТ 28147-89, а итоговое значение формируется как контрольное слово. Полученная имитопри-

ставка выводится после завершения обработки файла и может использоваться для контроля целостности передаваемых данных.

Реализованные функции и классы приведены в Приложении А (листинг программного кода).

Для проверки работоспособности программы был выбран текстовый файл, содержащий обычный читаемый текст. Проверка выполнялась отдельно для режимов простой замены, гаммирования и гаммирования с обратной связью. Для каждого режима сначала выполнялось шифрование входного файла, затем полученный файл расшифровывался с теми же параметрами. Дополнительно для результатов обработки вычислялась имитоприставка. Такой набор испытаний позволяет проверить как корректность базового блочного преобразования, так и корректность режимов работы алгоритма.

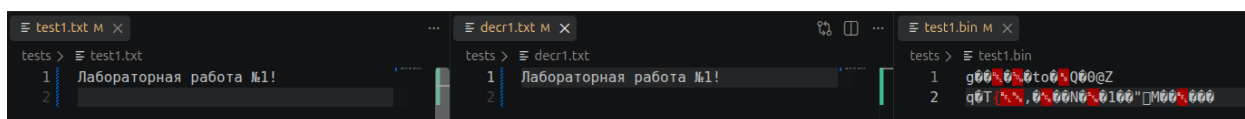


Рисунок 1.1 – Вывод программы при шифровании и расшифровании файла с использованием ГОСТ 28147-89

После расшифрования во всех проверенных режимах содержимое восстановленного файла совпадает с исходным. Это подтверждает корректность реализации процедур зашифрования и расшифрования. Полученная имитоприставка позволяет дополнительно контролировать целостность обработанных данных.

ВЫВОД

В ходе выполнения лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов на языке C++ с использованием алгоритма ГОСТ 28147-89. Реализованы предусмотренные заданием режимы простой замены, гаммирования и гаммирования с обратной связью, а также генерация имитоприставки для контроля целостности данных. Реализация включает разбор аргументов командной строки, чтение и запись файлов в бинарном режиме и применение обратного преобразования при расшифровании. Проведенное тестирование показало, что после расшифрования исходное содержимое файлов восстанавливается, а вычисление имитоприставки выполняется для полученных данных. Полученные результаты соответствуют поставленной в лабораторной работе задаче и позволяют на практике ознакомиться с принципами работы симметричного блочного шифра и его режимов.

ПРИЛОЖЕНИЕ А (обязательное) Листинг программного кода

Листинг А.1 – main.cpp

```
#include "gost_core.h"
#include <iostream>
#include <fstream>
#include <vector>

int main(int argc, char* argv[]) {
    if (argc < 5) {
        std::cout << "Usage: ./gost <mode: ECB|OFB|CFB> <enc|dec> <input>
<output>\n";
        return 1;
    }

    uint8_t key[32] = {0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF,
                      0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF,
                      0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF,
                      0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF};
    uint64_t iv = 0x1234567890ABCDEF;

    std::string mode = argv[1];
    bool encrypt = (std::string(argv[2]) == "enc");
    std::string input = argv[3];
    std::string output = argv[4];

    try {
        processGost(mode, encrypt, input, output, key, iv);

        std::ifstream outFile(output, std::ios::binary);
        std::vector<uint8_t> result(
            (std::istreambuf_iterator<char>(outFile)),
            std::istreambuf_iterator<char>())
        );

        GOST28147 cipher(key);
        std::cout << "MAC: " << std::hex
                  << cipher.calculateMAC(result) << std::endl;
        std::cout << "Successfully processed "
                  << input << " -> " << output << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Листинг А.2 – gost_core.cpp

```
#include "gost_core.h"
#include <cstring>
#include <fstream>
#include <algorithm>
#include <vector>

void GOST28147::round(uint32_t &N1, uint32_t &N2, uint32_t key) const {
    uint32_t S = N1 + key;
```

```

uint32_t transformed = 0;

for (int i = 0; i < 8; ++i) {
    transformed |= ((uint32_t)S_BOX[i]((S >> (4 * i)) & 0xF)
        << (4 * i));
}

transformed = (transformed << 11) | (transformed >> 21);

uint32_t nextN2 = N2 ^ transformed;
N2 = N1;
N1 = nextN2;
}

void GOST28147::encryptBlock(uint32_t &left, uint32_t &right) const {
    uint32_t n1 = left, n2 = right;

    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 8; ++j) {
            round(n1, n2, subkeys[j]);
        }

        for (int j = 0; j < 8; ++j) {
            round(n1, n2, subkeys[7 - j]);
        }

        left = n2;
        right = n1;
    }

    void GOST28147::encryptBlock16(uint32_t &left, uint32_t &right) const {
        uint32_t n1 = left, n2 = right;

        for (int i = 0; i < 2; ++i) {
            for (int j = 0; j < 8; ++j) {
                round(n1, n2, subkeys[j]);
            }

            left = n2;
            right = n1;
        }

        void GOST28147::decryptBlock(uint32_t &left, uint32_t &right) const {
            uint32_t n1 = left, n2 = right;

            for (int j = 0; j < 8; ++j) {
                round(n1, n2, subkeys[j]);
            }

            for (int i = 0; i < 3; ++i) {
                for (int j = 0; j < 8; ++j) {
                    round(n1, n2, subkeys[7 - j]);
                }
            }

            left = n2;
            right = n1;
        }

        uint32_t GOST28147::calculateMAC(const std::vector<uint8_t>& data) const {

```

```

    std::vector<uint8_t> b = data;

    while (b.size() % 8 != 0) {
        b.push_back(0);
    }

    uint32_t l = 0, r = 0;

    return l;
}

void processGost(const std::string& mode, bool encrypt,
                const std::string& input, const std::string& output,
                const uint8_t key[32], uint64_t iv) {
    GOST28147 cipher(key);

    std::ifstream inFile(input, std::ios::binary);
    std::ofstream outFile(output, std::ios::binary);

    std::vector<uint8_t> buffer(
        (std::istreambuf_iterator<char>(inFile)),
        std::istreambuf_iterator<char>()
    );

    if (mode == "ECB") {
        if (encrypt) {
            size_t padLen = 8 - (buffer.size() % 8);
            for (size_t i = 0; i < padLen; ++i) {
                buffer.push_back(static_cast<uint8_t>(padLen));
            }
        }

        for (size_t i = 0; i < buffer.size(); i += 8) {
            uint32_t left, right;

            std::memcpy(&left, &buffer[i], 4);
            std::memcpy(&right, &buffer[i + 4], 4);

            if (encrypt)
                cipher.encryptBlock(left, right);
            else
                cipher.decryptBlock(left, right);

            std::memcpy(&buffer[i], &left, 4);
            std::memcpy(&buffer[i + 4], &right, 4);
        }

        if (!encrypt) {
            uint8_t pad = buffer.back();

            if (pad > 0 && pad <= 8) {
                bool valid = true;

                for (size_t i = 0; i < pad; ++i) {
                    if (buffer[buffer.size() - 1 - i] != pad)
                        valid = false;
                }

                if (valid)
                    buffer.resize(buffer.size() - pad);
            }
        }
    }
}

```

```

    }
    else if (mode == "OFB") {
        uint64_t reg = iv;

        for (size_t i = 0; i < buffer.size(); i += 8) {
            uint32_t left = reg & 0xFFFFFFFF;
            uint32_t right = reg >> 32;

            cipher.encryptBlock(left, right);

            uint64_t gamma =
                (static_cast<uint64_t>(left)) |
                (static_cast<uint64_t>(right) << 32);

            for (size_t j = 0; j < 8 && (i + j) < buffer.size(); ++j) {
                buffer[i + j] ^= (gamma >> (8 * j)) & 0xFF;
            }

            reg = gamma;
        }
    }
    else if (mode == "CFB") {
        uint64_t reg = iv;

        for (size_t i = 0; i < buffer.size(); i += 8) {
            uint32_t left = reg & 0xFFFFFFFF;
            uint32_t right = reg >> 32;

            cipher.encryptBlock(left, right);

            uint64_t gamma =
                (static_cast<uint64_t>(left)) |
                (static_cast<uint64_t>(right) << 32);

            uint64_t block = 0;
            std::memcpy(&block, &buffer[i],
                std::min(size_t(8), buffer.size() - i));

            uint64_t encrypted = block ^ gamma;

            if (encrypt) {
                std::memcpy(&buffer[i], &encrypted,
                    std::min(size_t(8), buffer.size() - i));
                reg = encrypted;
            }
            else {
                reg = block;
                std::memcpy(&buffer[i], &encrypted,
                    std::min(size_t(8), buffer.size() - i));
            }
        }
    }

    outFile.write(reinterpret_cast<char*>(buffer.data()), buffer.size());
}

```